# 智安工坊

# AI 安全开发协作平台

AI WEB 网页开发挑战赛技术文档

CodeGuard 代码安全审计引擎升级版

---

项目定位：面向大学生和教师的本地 AI Web 安全开发协作平台。

文档版本：传智杯 AI WEB 参赛提交版

说明：本文档只描述当前实际实现的功能，不把未完成的 Agent、RAG 或数据库能力写成已实现功能。

# 目录


- 一、项目简介
- 二、需求分析
- 三、总体框架设计
- 四、技术选型
- 五、核心功能设计
- 六、AI 技术应用方法
- 七、数据与状态设计
- 八、核心功能实现
- 九、测试情况
- 十、项目创新点
- 十一、应用价值
- 十二、未来规划

# 一、项目简介

在课程作业、实验项目和毕业设计中，代码经常会经历多人协作、压缩传递、公开仓库提交和答辩展示。一个看似普通的配置项，可能包含 API Key、数据库密码、JWT Token 或真实个人信息。传统教学中，学生往往只在出现事故后才意识到敏感信息泄露的影响。智安工坊将安全检查前移到代码提交之前，通过网页化、可视化和 AI 辅助解释，让学生能够看见风险、理解风险并完成复核。

项目名称为“智安工坊 - AI 安全开发协作平台”，内部保留 CodeGuard 代码安全审计引擎。系统提供本地网页入口，后端使用 Flask 提供扫描接口、状态接口、报告接口和 DeepSeek 调用接口，前端使用原生 HTML、CSS 和 JavaScript 完成交互。

## 项目目标

- 帮助学生在提交代码前发现敏感信息泄露线索。
- 通过风险等级、置信度和修复示例降低安全学习门槛。
- 为教师提供可复核、可归档的审计报告和整改记录。
- 在调用大模型时进行脱敏，避免把真实凭据直接发送给外部服务。

## 当前实现边界

系统只做风险发现、解释、修复建议和人工复核，不自动修改用户源码，不验证密钥是否真实有效，也不替代正式的渗透测试、密钥轮换和权限治理。

# 二、需求分析

## 2.1 用户需求

- 学生希望通过简单的粘贴或上传操作完成代码安全检查。
- 学生需要知道风险具体位于哪个文件、哪一行，而不是只看到一段抽象提示。
- 教师需要看到风险类型、风险等级、复核状态和整改前后变化。
- 课程作业和毕业设计需要一份可下载、可留档的 PDF 结果。
- 调用 AI 时需要保护真实密钥，且在网络或额度不可用时仍能完成基本使用。

## 2.2 功能需求

| 需求类别 | 具体需求 | 实现结果 |
| --- | --- | --- |
| 输入 | 粘贴代码或上传 ZIP | 支持文本接口和 ZIP 接口，限制大小并过滤路径穿越 |
| 检测 | 识别多类敏感信息 | 规则信号、上下文、文件类型和高熵特征组合评分 |
| 理解 | 解释风险并给修复示例 | 内置建议 + 可选 DeepSeek 建议 |
| 复核 | 确认、误报、已修复等状态 | 前端下拉选择并同步后端当前扫描结果 |
| 归档 | 导出报告和查看历史 | ReportLab PDF 和运行期间扫描历史对比 |

# 三、总体框架设计

## 3.1 系统架构

| 层次 | 模块 | 职责 |
| --- | --- | --- |
| 浏览器层 | HTML / CSS / JavaScript | 输入代码、上传 ZIP、展示风险、触发 AI 和 PDF 下载 |
| 接口层 | Flask 路由 | 处理扫描、状态更新、历史查询、报告和 AI 请求 |
| 检测层 | CodeGuard Scanner | 识别风险、计算分数、生成置信度和修复建议 |
| 洞察层 | Project Insights | 依赖清单分析、Git 历史证据和扫描结果对比 |
| AI 层 | DeepSeek Adapter | 发送脱敏上下文，解析中文建议，失败时回退 |
| 报告层 | ReportLab | 生成中文 PDF 审计报告 |

## 3.2 主要数据流

- 用户从网页输入代码或上传 ZIP。
- Flask 校验输入，读取支持的文本文件，过滤二进制文件、构建目录和虚拟环境。
- CodeGuard 引擎逐行匹配规则并进行组合评分，生成 Finding 对象。
- 项目洞察模块分析依赖清单和上传包中的 Git 证据。
- 前端以风险列表、代码行高亮和指标卡片展示结果。
- 用户可以选择人工状态、生成 AI 建议、查看历史对比或导出报告。

## 3.3 安全边界

上传 ZIP 使用路径检查，跳过绝对路径和包含 .. 的路径；单文件和总文件数量有限制；DeepSeek 请求只使用 masked_line 等脱敏上下文；真实 API Key 只从本地 .env 读取，并通过 .gitignore 排除。

# 四、技术选型

| 技术 | 版本或形式 | 选择原因 |
| --- | --- | --- |
| Python | 3.12 | 便于教学、测试和跨平台部署 |
| Flask | 3.x | 接口清晰，适合轻量本地 Web 原型 |
| HTML/CSS/JavaScript | 原生实现 | 无需额外前端构建工具，浏览器即可运行 |
| ReportLab | 4.x | 本地生成中文 PDF，不依赖在线文档服务 |
| DeepSeek API | Chat API | 用于教育性解释和项目级整改计划 |
| pytest | 8.x | 验证检测、评分、依赖分析和接口行为 |
| Git | 本地仓库 + GitHub | 保存细粒度开发记录，支持评审复现 |

## 数据库说明

当前版本没有使用数据库。扫描结果、人工复核状态和扫描历史保存在 Flask 进程内存中，避免学生上传代码被长期落盘。后续如果需要多人协作和长期归档，可增加 SQLite 或 PostgreSQL，但当前提交版不把不存在的数据库写成已实现功能。

# 五、核心功能设计

## 5.1 多入口代码审计

文本入口适合课堂中快速演示单个配置片段；ZIP 入口适合对课程项目或毕业设计进行多文件扫描。系统只读取支持的文本后缀，并跳过 node_modules、.venv、build、dist 等目录，减少无关内容和误报。

## 5.2 风险分级与定位

每条 Finding 包含文件、行号、列号、风险类别、规则名称、分数、置信度、证据、脱敏行和修复示例。评分会结合上下文关键词、赋值结构、配置文件类型、凭据长度和占位符判断，最终映射为严重、高、中、低四级。

## 5.3 人工复核闭环

检测不是结论。系统提供待确认、已确认、误报、已修复和忽略本次五种状态。学生可以先确认风险，再轮换凭据、改用环境变量并重新扫描；教师可以在报告中查看整改状态。

## 5.4 项目洞察

| 洞察项 | 当前实现 | 使用价值 |
| --- | --- | --- |
| 依赖清单 | 解析 requirements.txt、package.json、pom.xml 等 | 建立依赖清单，提示版本范围过宽 |
| Git 证据 | 识别 ZIP 中的 .git 并读取提交摘要 | 提示开发过程是否可追溯 |
| 扫描历史 | 保留最近 12 次运行结果 | 对比风险数量和安全评分变化 |
| 整改计划 | 按风险分数排序生成内置或 AI 计划 | 把多个问题组织成可执行任务 |

# 六、AI 技术的应用方法

## 6.1 调用流程

- 本地规则引擎先完成检测，DeepSeek 不负责替代本地扫描。
- 用户点击单条建议后，系统组织文件名、行号、风险类别和脱敏代码行。
- 后端通过 DeepSeek Chat API 请求 JSON 格式的 advice 和 example。
- 系统解析返回内容并显示在当前风险卡片中，不写回源代码。
- 项目级整改计划最多发送排序后的脱敏风险摘要，减少调用内容和额度消耗。

## 6.2 隐私与可用性

真实敏感值在发送前由本地扫描器替换为类别和部分掩码。Key 不写入仓库，也不进入报告。若 .env 未配置、网络连接失败、接口返回错误或额度不足，系统自动使用内置中文建议，使本地检测功能保持可用。

## 6.3 实现边界

当前 AI 能力是按需调用的建议生成，不宣称具备自主 Agent、RAG 知识库或自动修复能力。后续可以在不改变安全边界的前提下增加本地安全知识库和可审计的多步骤任务编排。

# 七、数据与状态设计

## 7.1 核心数据对象

| 对象 | 主要字段 | 说明 |
| --- | --- | --- |
| ScanResult | scan_id、files、findings、source_name | 一次扫描的完整结果 |
| Finding | file_name、line_number、category、score、confidence | 单条风险线索和整改信息 |
| DependencyInsight | manifest、name、version、status | 依赖清单卫生提示 |
| GitEvidence | commit_count、author_count、recent_commits | 提交历史证据摘要 |
| HistoryEntry | scan_id、created_at、summary | 运行期间保存的扫描摘要 |

## 7.2 为什么当前不使用数据库

项目主要服务于本地课程和比赛演示，代码不应被默认长期保存。内存模型可以清晰展示功能闭环，减少数据库账号和部署依赖，也让学生更容易复现。产品化阶段再根据隐私和协作需求选择数据库。

## 7.3 关键 API

| 接口 | 方法 | 用途 |
| --- | --- | --- |
| /api/scan/text | POST | 扫描粘贴代码 |
| /api/scan/zip | POST | 扫描 ZIP 项目 |
| /api/findings/{id}/status | POST | 更新人工复核状态 |
| /api/findings/{id}/advice | POST | 获取单条 AI 或内置建议 |
| /api/project/remediation-plan | POST | 获取项目级整改计划 |
| /api/history/compare | GET | 对比两次扫描结果 |
| /api/report.pdf | GET | 导出当前 PDF 报告 |

# 八、核心功能实现

## 8.1 检测规则与组合评分

规则以类别、名称、基础分数、正则信号、修复建议和示例代码组成。扫描器会把上下文关键词、赋值结构、文件类型、占位符和字符串长度纳入评分，并在同一文件同一行同一类别出现多次时保留更高风险信号。

## 8.2 ZIP 安全处理

系统拒绝超出大小限制的 ZIP，过滤绝对路径和 .. 路径，限制文件数量和单文件大小，并尝试 UTF-8 与 GB18030 解码。对于 Git 历史，只有压缩包明确包含 .git/HEAD 时才读取提交摘要。

## 8.3 报告生成

ReportLab 生成中文 PDF，内容包括扫描来源、扫描时间、文件和代码行数、安全评分、风险等级统计、脱敏代码行、修复建议、整改记录、依赖洞察和 Git 证据。

### 8.4 启动与复现

```
py -m venv .venv .\.venv\Scripts\Activate.ps1 python -m pip install -r requirements.txt python run.py
```

## 九、测试情况

项目使用 pytest 编写自动化测试，并通过 Flask 测试客户端完成接口烟测。测试重点不是只验证页面能打开，还验证风险识别、评分、过滤、项目洞察、历史和报告流程。

| 测试类别 | 测试内容 | 结果 |
| --- | --- | --- |
| 检测规则 | API Key、数据库密码、邮箱、手机号、JWT 等 | 通过 |
| 占位符处理 | 示例 Key 不应被判定为同等级真实凭据 | 通过 |
| 目录过滤 | node_modules 等目录不进入结果 | 通过 |
| 评分 | 干净代码满分，存在风险时扣分 | 通过 |
| 依赖分析 | 识别清单并提示宽版本范围 | 通过 |
| Git 证据 | 无 .git 历史时给出明确提示 | 通过 |
| 接口烟测 | 扫描、状态、历史、Git 证据和 PDF | 通过 |
| AI 调用 | 脱敏后调用 DeepSeek，失败时回退内置建议 | 已验证 |

## 十、项目创新点

- 把敏感信息检查从抽象的安全术语转化为学生能够直接理解的代码行、风险等级和示例代码。
- 采用本地规则检测 + 脱敏大模型建议的组合方式，兼顾响应速度、可解释性和隐私边界。
- 引入人工复核状态，避免把规则命中直接等同于漏洞结论。
- 把依赖清单、Git 开发证据和扫描历史纳入同一工作台，形成从发现到归档的闭环。
- 在网络、Key 或额度不可用时仍能使用本地检测和内置建议，降低课堂演示失败风险。

## 十一、应用价值

对学生而言，系统适合作业提交前自查、毕业设计阶段性检查和代码安全意识训练；对教师而言，可以用演示数据讲解密钥管理、人工复核和整改闭环，并将 PDF 报告作为课程过程材料；对项目团队而言，Git 记录和测试结果可以帮助建立可复现的开发过程。

平台也可作为实验室内部的轻量安全入口，后续接入企业级密钥管理、依赖漏洞数据库、代码仓库 Webhook 和权限系统后，可以扩展到更完整的开发安全流程。

## 十二、未来规划

- 增加 SQLite 持久化和项目成员权限，支持长期课程归档。

- 建立本地可审计的安全知识库，用于检索修复规范和课堂案例。
- 增加依赖漏洞数据库同步和更细的供应链风险分析。
- 支持 GitHub/Gitee 仓库授权扫描，并在提交前自动触发检查。
- 增加更严格的测试数据集、误报反馈和规则版本管理。
- 在条件允许时部署公开演示地址，但仍保持密钥脱敏和最小化数据留存。

## 结语

智安工坊的核心不是把安全审计做成一个复杂的黑盒，而是让学生在一次次扫描、复核和整改中形成正确的开发习惯：密钥不写进源码，风险要看证据，修复要留记录，提交前要再检查。

---

文档结束